S6- Bitácora de Cambios (Changelog)

INTEGRANTES:

- ALFREDO MALDONADO
- LUIS RIOS
- CRISTHIAN VELEZ

Fecha: 25 de FEB. de 25

Contenido

1. Objetivo General 3

1.1 Objetivos Específicos 3

2.1 Bitácora de Cambios (Changelog) 4

[v1.0] - Intento Inicial 4

[v1.1] - Implementación Streamlit + LocalTunnel 4

[v1.2-broken] - Falla en Preprocesamiento Matemático 5

[v1.2-stable] - Baseline Funcional 5

3. Bibliografía Seleccionada 7

1. Objetivo General

Desarrollar un prototipo de sistema de segmentación de clientes utilizando algoritmos de aprendizaje no supervisado (K-Means) para identificar perfiles de consumo diferenciados y optimizar la efectividad de las campañas de marketing en cadenas comerciales del sector retail, permitiendo una transición de estrategias masivas a acciones personalizadas basadas en datos transaccionales, bajo el marco legal de protección de datos en el Ecuador.

1.1 Objetivos Específicos

1. **Ejecutar el proceso de extracción, limpieza y preprocesamiento de datos** del historial transaccional en un periodo de dos semanas, calculando las métricas de Recencia, Frecuencia y Valor Monetario (RFM) para construir la base de conocimiento del modelo.y Determinar el número óptimo de clústeres *(k)* mediante la aplicación de técnicas de validación estadística como el Método del Codo y el Coeficiente de Silueta en un plazo de dos semanas, asegurando la cohesión interna y separación de los segmentos identificados.
2. **Implementar y entrenar el algoritmo K-Means** utilizando Python y Scikit-learn durante un ciclo de cinco semanas, logrando una clasificación de la base de clientes en segmentos accionables para el negocio (VIP, Leales, Ocasionales, En Riesgo O Similares).
3. **Construir un dashboard interactivo de visualización** mediante Streamlit en un periodo de tres semanas, que integre los resultados del modelo y permita a los gerentes de marketing explorar las características de cada segmento para el diseño de campañas. y Evaluar el impacto potencial del prototipo mediante la comparación de métricas históricas de conversión contra las proyecciones del modelo.

2.1 Bitácora de Cambios (Changelog)

Esta bitácora registra las iteraciones, errores críticos y correcciones aplicadas al modelo baseline.

[v1.0] - Intento Inicial

- Acción: Desarrollo de interfaz inicial usando Dash/Flask.
- Problema: Bloqueos continuos de puertos en Google Colab. Imposibilidad de renderizar la aplicación web de forma estable en el entorno de pruebas.
- Resolución: Se decide cambiar la librería de frontend a Streamlit por su mayor compatibilidad y facilidad de prototipado.
- Intento Fallido con ipywidgets (Beta_V1_0.pdf): *Enfoque: Uso de ipywidgets directamente en la celda de Colab.
- Error: Problemas graves de renderizado, sincronización de estado de los botones y una interfaz poco profesional (dependencia de HTML/CSS inyectado en celdas).

[v1.1] - Implementación Streamlit + LocalTunnel

- Acción: Reescritura del frontend a Streamlit. Uso de localtunnel vía npm para exponer la aplicación.
- Problema: Pantallas de carga infinitas en el túnel. Errores ERR_NGROK_121 al mezclar dependencias, y problemas de compatibilidad (URLs de descarga caídas para v3 de ngrok reportadas en logs).
- Resolución: Abandono definitivo de localtunnel. Transición estricta a Ngrok requiriendo Authtoken manual para garantizar el uptime de la demostración.
- Caos de Despliegue y Túneles (Beta_V1_1.pdf): * Enfoque: Migración a Streamlit (app.py). Intento de exposición vía localtunnel y posteriormente CLI de ngrok.
- Error 1: localtunnel falló debido a problemas de dependencias npm y expiración de sesión.

- Error 2: Instalación manual de ngrok falló por errores de sintaxis en bash, descarga de versión v2 obsoleta (Error ERR_NGROK_121).
- Error 3: Intentos de descarga de ngrok v3 fallaron reiteradamente por URLs inestables (Errores HTTP 404). Requirió intervención manual constante, rompiendo la automatización.

[v1.2-broken] - Falla en Preprocesamiento Matemático

- Acción: Despliegue exitoso de la interfaz gráfica vía Ngrok. Se introduce el archivo de prueba retail_sales_dataset.csv.
- Problema (Crítico): Crash total del backend al ejecutar la agregación RFM.
    - Error Trace: TypeError: datetime64 type does not support sum operations.
    - Causa: El usuario mapeó incorrectamente las columnas, o el dataset tenía datos sucios. El método .agg() intentó ejecutar una suma matemática sobre una fecha, evidenciando la ausencia de sanitización y coerción de tipos en la función calculate_rfm().
- Resolución: Refactorización inmediata del motor lógico. Se añadieron barreras de tipo: pd.to_datetime(..., errors='coerce') y pd.to_numeric(..., errors='coerce') antes de la agrupación, además de validaciones dropna() para registros huérfanos.

[v1.2-stable] - Baseline Funcional

- Acción: Liberación de la versión estable del prototipo.
- Agregados:
    - Coerción de tipos segura.
    - Implementación de gráficos interactivos 3D (Plotly) para visualizar Recency, Frequency y Monetary.
    - Mapeo del modelo K-Means a etiquetas de negocio ("VIP", "Leales", "En Riesgo", "Perdidos").
    - Panel de recomendaciones tácticas habilitado.
- Estado: Prototipo mínimo viable (MVP) aceptado para evaluación inicial de tesis.

- Estabilización del MVP (Beta_V1_2.pdf): * Solución: Uso de la librería de Python pyngrok, eliminando la necesidad de descargar binarios manualmente.
- Resultado: Despliegue exitoso. Interfaz de Streamlit estable y generación correcta de gráficos (Plotly 3D y barras). Nota: Se introdujo el error lógico de segmentación por cuartiles mencionado en el informe.

[v1.4 / v1.3] – Actualización de K dinámico.

- K dinámico (Método Codo + Silueta): Se integró el cálculo automatizado de inercias y Silhouette Scores iterando desde K=2 hasta K=10, seleccionando el K óptimo topológico y eliminando el hardcoding de K=4.
- Mapeo algorítmico de centroides: Eliminación total de la lógica dura if/else por cuantiles. Introducción de assign_dynamic_segments, la cual estandariza y promedia (Recencia, Frecuencia, Monto) midiendo los centroides reales descubiertos por el K-Means para clasificar VIP, Leales, Riesgo y Perdidos basándose en distancias matemáticas verdaderas.

[v1.5] – Revisión de streamlit

- Fijación de Estado UI (Session State): Se implementó st.session_state.ejecutado para corregir la falla crítica del ciclo de vida de Streamlit, la cual reseteaba el dashboard a blanco al interactuar con el selectbox de recomendaciones de marketing.

[v1.6] – Actualización CSV y parseo

- Resiliencia en parseo CSV: Se reescribió la función load_dataset con un bloque try-except. Si el archivo delimitado por punto y coma (;) u otro delimitador corrompe la lectura estándar, el puntero se reinicia (seek(0)) y fuerza la lectura evadiendo líneas conflictivas (on_bad_lines='skip').
- Autodetección de features: Se programaron heurísticas (next(enumerate(...))) para pre-seleccionar automáticamente las columnas de ID, Fecha y Monto en los menús desplegables.

[v1.7] - 25/02/2026 - (Versión Actual)

Evidencias de control de versiones y correcciones aplicadas:

- Manejo de memoria en ingesta: Se añadió el parámetro low_memory=False en las lecturas de pd.read_csv para eliminar los DtypeWarning en datasets con tipos de datos mixtos.
- Procesamiento de fechas robusto: Se ajustó la coerción de la columna de fechas implementando dayfirst=True y format='mixed' en pd.to_datetime(), previniendo excepciones de parseo por inversión de días/meses.
- Corrección UI Plotly: Se eliminó el parámetro deprecado use_container_width=True en la llamada de st.plotly_chart() para evitar warnings de la API de Streamlit.
- Formato Financiero: Integración de formato de moneda ($,.2f) directamente en el hover de los tooltips y los ejes Z de la gráfica Plotly 3D.
- Escalabilidad de visualización tabular: Se inyectó pd.set_option("styler.render.max_elements", 5000000) en la configuración global para soportar bases de datos masivas, evitando el límite restrictivo por defecto de 262,144 celdas.
- Integración de drill-down: Adición de un DataFrame interactivo al final de la interfaz, que filtra y expone el histórico transaccional detallado asociado exclusivamente al clúster (segmento) seleccionado en tiempo real, renderizando los montos en formato dólar.

3. Bibliografía Seleccionada

1. Paramita, A. S., & Hariguna, T. (2024). Comparative Studies on Clustering Algorithms K-Means and DBSCAN. Journal of Digital Market and Digital Currency. [1]
2. Gan, W., Zhou, P., Wan, S., & Zeng, J. (2025). RFMI-based Customer Segmentation with K-means. 2024 IEEE International Conference on Big Data. [2]
3. John, J. M., Shobayo, O., & Ogunleye, B. (2025). The RFM-A Model: Combining Behavioral and Demographic Data for Segmentation. The Journal of Supercomputing. [3]
4. Srivastava, P., & Saxena, D. S. (2025). Enhanced Customer: A Comparative

Analysis of Leading Clustering Algorithms. International Journal of Novel Research and Development, 10(4). [4]

5. Nugroho, B. I., Rafhina, A., & Ananda, P. S. (2024). Customer segmentation in sales transaction data using k-means clustering algorithm. Journal of Intelligent Decision Support System, 7(2). [5]
6. Hidayat, T., Ruldeviyani, A., & Aditama, A. R. (2024). Optimal Use of Customer Segmentation using RFM and K-means. Scientific Journal of Informatics, 11(1). [6]
7. Lai, M. T. (2025). Comparison of K-Means and DBSCAN Algorithms for Customer Segmentation. ResearchGate. [7]
8. Lewaaelhamd, I. (2024). Customer segmentation using machine learning model: An application of RFM analysis. Journal of Data Science and Intelligent Systems, 2, 29–36. [8]
9. Nithyashree, R. (2024). Decoding Decision Support Systems: Empowering Malls with Customer Segmentation. Medium Data Science. [9]
10. Ogunleye, B. (2024). An Exploration of Clustering Algorithms for Customer Segmentation in the UK Retail Market. Analytics, 2(4), 809-823. [10]